PES UNIVERSITY | DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

LAB 7 - FUZZ TESTING FOR INPUT VALIDATION BUGS

Automated Defect Discovery, Minimization & Defensive Hardening Using Hypothesis

Course:	Software Engineering Laboratory (UE23CS352AA)
Lab Title:	Lab 7: Fuzz Testing for Input Validation Bugs
Assigned System:	Elevator Dispatch System (Module #38)
Student Name:	**Anish A Kunder**
SRN:	**PES1UG24AM343**
Repository:	https://github.com/planetai05/L6_L7_38_elevator_dispatch
Branch:	lab6-lab7-solutions
Date of Submission:	November 2026

Executive Summary:

This report documents the property-based fuzz testing analysis and defensive remediation of the assigned Elevator Dispatch module. Using Python's **Hypothesis** testing framework, randomized and malformed data streams were synthesized across type spaces, boundaries, and encodings. Hypothesis discovered and minimized **10 genuine defects and crashes** (AttributeError, ValueError, ZeroDivisionError, and negative telemetry) in the original code. The application was then defensively refactored with deterministic input validation and bounds guards without blanket exception handling, achieving a **flawless 100% pass rate (14/14 tests)**.

1. Introduction & Lab Objective

Fuzz testing (fuzzing) is an automated software quality assurance and vulnerability discovery methodology where programs are bombarded with semi-random, malformed, or unexpected inputs to observe failure states. In contrast to traditional unit tests that only validate scenarios envisioned by the developer, fuzz testing explores unanticipated combinations, boundary values, and malicious payloads.

Lab Learning Objectives:

- Apply property-based fuzz testing using Python's **Hypothesis** library with @given decorators.
- Uncover edge-case crashes, boundary exceptions, and type errors in the unhardened application.
- Analyze test failure minimization and automatic input shrinking performed by Hypothesis.
- Defensively harden all application routines using explicit input validation without blanket exception handling.

2. Description of the Original Application

The target application (processor.py / elevator_dispatch.py) processes elevator requests, floor travel calculations, and passenger load telemetry. The original code assumed ideal inputs without type checks or guards.

3. Explanation of the Three Functions

Function	Signature & Expected Input	Expected Behavior & Formula
`parse_request`	text: str Format: 'START->END' (e.g. '5->9')	Parses origin and destination floors into an integer tuple (start, end). Must reject malformed strings.
`total_travel`	floors_csv: str Format: '1,5,3,9'	Calculates cumulative vertical travel: $\sum \lvert floor_{i+1} - floor_i \rvert$. Returns integer ≥ 0.
`load_percent`	text: str Format: 'LOAD of CAPACITY' (e.g. '450 of 630')	Evaluates integer load percentage: (load * 100) // capacity. Requires non-negative load and capacity > 0.

4. Fuzz Testing & Hypothesis Methodology

Hypothesis verifies that high-level specifications (properties) hold across an input space. When an exception or assertion failure occurs, Hypothesis performs **counterexample shrinking**, iteratively pruning characters or reducing integer magnitudes until the smallest possible failing input is identified (e.g. shrinking an arbitrary string crash down to '' or None).

5. Hypothesis Strategies Used

- st.text() | st.none(): Generates arbitrary Unicode strings, empty strings, emojis, and null references.
- st.integers(): Generates negative numbers, zero, large integers ($\pm 10^5$), and boundary values.
- st.lists(st.integers()): Generates floor sequences from empty lists up to 30 elements.
- st.one_of(...): Tests foreign types (booleans, floats, lists) to enforce strict type verification.

6. Initial Application Execution

The original, unhardened elevator dispatch script was initially executed using valid sample inputs ('5->9', '1,5,3,9', '450 of 630'). Under standard conditions, the script produced expected outputs as captured in Figure 1 below:

```
d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7>python elevator_dispatch.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5->9
From and To: (5, 9)

==== Running total_travel ====
Enter floors visited as CSV (e.g. 1,5,3,9): 1,5,3,9
Floors Travelled: 12

==== Running load_percent ====
Enter load as 'X of Y' (e.g. 450 of 630): 450 of 630
Load Percent: 71

d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7>
```

Figure 1: *Initial execution of the original elevator dispatch application with nominal interactive inputs.*

7. Initial Failing Test Results

Running the comprehensive Hypothesis property test suite against the unhardened code resulted in immediate failure. Hypothesis detected **10 failing tests and 4 passing tests** (**10 failed, 4 passed in 0.99s**), exposing fatal crashes across every unvalidated function, as shown in the terminal capture in Figure 2:

```
d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7 soln>powershell -NoProfile -Command "Get-Content -Encoding Unicode 'evidence\hypothesis_failing_tests.txt' | Select-String -Pattern
'ValueError:|AttributeError:|ZeroDivisionError:|10 failed, 4 passed' | Select-Object -Last 12"

E           AssertionError: total_travel crashed on non-string type <class 'int'> with AttributeError: 'int' object has no attribute 'split'
E       ValueError: invalid literal for int() with base 10: ''
    | ValueError: not enough values to unpack (expected 2, got 1)
    | AssertionError: load_percent crashed with: ValueError: not enough values to unpack (expected 2, got 1)
    | AttributeError: 'NoneType' object has no attribute 'split'
    | AssertionError: load_percent crashed with: AttributeError: 'NoneType' object has no attribute 'split'
E       AttributeError: 'int' object has no attribute 'split'
E           AssertionError: load_percent crashed on non-string type <class 'int'> with AttributeError: 'int' object has no attribute 'split'
    | ZeroDivisionError: integer division or modulo by zero
    | AssertionError: load_percent crashed with ZeroDivisionError: integer division or modulo by zero
FAILED test_processor.py::test_total_travel_valid_property - ValueError: inva...
====================== 10 failed, 4 passed in 0.99s =======================


d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7 soln>
```

Figure 2: *Terminal evidence displaying Hypothesis discovering and shrinking 10 test failures on unhardened code.*

8. Bugs Discovered and Their Explanations

Hypothesis automatically shrunk counterexamples to identify 10 distinct failure modes in the original implementation:

#	Function	Shrunk Input	Observed Exception	Root Cause & Analysis
1	`parse_request`	`None, 0`	AttributeError	Calling .split() on non-string types raises unhandled attribute error.
2	`parse_request`	`''`	ValueError	Unpacking string without delimiter fails (expected 2 values, got 1).
3	`parse_request`	`'->'`	ValueError	Splitting yields empty tokens; int('') fails with invalid literal.
4	`total_travel`	`None, 0`	AttributeError	Non-string parameter lacks .split() method.
5	`total_travel`	`''`	ValueError	Splitting empty string yields [''], crashing on int('').
6	`total_travel`	`',', '1,,3'`	ValueError	Consecutive or trailing commas generate empty tokens that cannot parse as integers.
7	`load_percent`	`None, [1]`	AttributeError	Non-string parameter crashes immediately on .split().
8	`load_percent`	`''`	ValueError	Missing ' of ' delimiter triggers unpacking error.
9	`load_percent`	`'10 of 0'`	ZeroDivisionError	Car capacity of 0 kg causes integer division by zero.
10	`load_percent`	`'-10 of 100'`	**Logic Violation**	Negative passenger weight yields invalid negative percentage (-10%).

9. Fixes Implemented

All 10 defects were remediated in processor.py via defensive programming without blanket exception handling:

- **Explicit Integer Validator:** Implemented _is_integer(s) to deterministically verify whether a string is non-empty, contains an optional leading + or - sign, and consists strictly of digits.
- **Type & Null Guards:** Added isinstance(..., str) at the entry of each function, safely returning None (or 0) on foreign types.
- **Delimiter Verification:** Validated that delimiters occur exactly once, guaranteeing valid binary unpacking.
- **Physical Domain Safeguards:** Checked that capacity > 0 (eliminating ZeroDivisionError) and load ≥ 0.
- **Deterministic Return Protocol:** Invalid inputs return None for parsing/percentage routines and 0 for travel distance.

10. Fixed Application Execution

The hardened application was re-run with valid interactive inputs. Intended business logic was completely preserved:

```
d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7 soln>python processor.py
==== Running parse_request ====
Enter a request as FROM->TO (e.g. 5->9): 5->9
From and To: (5, 9)

==== Running total_travel ====
Enter floors visited as CSV (e.g. 1,5,3,9): 1,5,3,9
Floors Travelled: 12

==== Running load_percent ====
Enter load as 'X of Y' (e.g. 450 of 630):       450 of 630
Load Percent: 71

d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7 soln>
```

Figure 3: *Execution of the fixed elevator dispatch application verifying preserved functionality.*

11. Final Successful Hypothesis Tests

The complete Hypothesis test suite was executed against the hardened code. All **14 property-based tests passed flawlessly (14 passed in 1.28s)** across thousands of synthesized inputs, confirming crash resilience and mathematical correctness:

```
d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7 soln>python -m pytest test_processor.py -v
============================= test session starts ==============================
platform win32 -- Python 3.12.4, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\Aryan Ashok\AppData\Local\Programs\Python\Python312\python.exe
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7 soln
plugins: hypothesis-6.170.15
collected 14 items

test_processor.py::test_parse_request_no_crash PASSED                    [  7%]
test_processor.py::test_parse_request_invalid_types_no_crash PASSED      [ 14%]
test_processor.py::test_parse_request_valid_property PASSED              [ 21%]
test_processor.py::test_parse_request_malformed_returns_none PASSED      [ 28%]
test_processor.py::test_total_travel_no_crash PASSED                     [ 35%]
test_processor.py::test_total_travel_invalid_types_no_crash PASSED       [ 42%]
test_processor.py::test_total_travel_valid_property PASSED               [ 50%]
test_processor.py::test_total_travel_reversal_symmetry PASSED            [ 57%]
test_processor.py::test_total_travel_triangle_inequality PASSED          [ 64%]
test_processor.py::test_load_percent_no_crash PASSED                     [ 71%]
test_processor.py::test_load_percent_invalid_types_no_crash PASSED       [ 78%]
test_processor.py::test_load_percent_valid_property PASSED               [ 85%]
test_processor.py::test_load_percent_zero_or_negative_capacity_no_crash PASSED [ 92%]
test_processor.py::test_load_percent_negative_load_no_crash PASSED       [100%]

============================== 14 passed in 1.28s ==============================

d:\Sem 5\SE Lab\Lab_6_7\L6_L7_38_elevator_dispatch\lab7 soln>
```

Figure 4: *Terminal evidence demonstrating 100% passing rate (14 passed) on the hardened application.*

12. Conclusion

Fuzz testing with Hypothesis demonstrated its power in finding vulnerabilities that manual test cases easily miss, such as zero-capacity division, non-string injections, and delimiter misconfigurations. By combining automated counterexample shrinking with disciplined defensive programming, all edge cases were safely guarded, transforming fragile scripts into robust, fault-tolerant elevator control software.